

Travaux pratiques : Estimation de Profondeur

Exercice 1 :

Deux caméras identiques sont séparées par une ligne de base $B = 10$ cm. Leur longueur focale est $f = 500$ px. Un point de la scène produit une disparité $d = 20$ px.

La profondeur z est donnée par :

$$z = (f \times B) / d$$

Complétez le tableau ci-dessous en calculant z pour chaque valeur de disparité :

f (px)	B (cm)	d (px)	z (cm)
500	10	5	
500	10	10	
500	10	25	
600	15	30	

Exercice 2 :

Une caméra possède une longueur focale $f = 50$ mm. Le capteur est déplacé à différentes positions s . En utilisant la loi de Gauss des lentilles, calculez la profondeur objet o pour chaque position du capteur.

Loi de Gauss des lentilles :

$$1/f = 1/s + 1/o \Rightarrow o = (s \times f) / (s - f)$$

Complétez le tableau ci-dessous :

f (mm)	s (mm)	$s - f$ (mm)	o (m)
50	51.25	1.25	2.05
50	52.00	2.00	
50	55.00	5.00	
50	60.00	10.00	

50	100.0	50.00	
----	-------	-------	-------

Exercice 3 :

La **Mesure de Netteté (FM)** en un pixel est la somme des réponses au carré du Laplacien Modifié dans une petite fenêtre. Elle est élevée lorsque la région est nette.

Un capteur est déplacé à 5 positions. Les valeurs de FM mesurées pour une région donnée sont :

$s_1 = 50 \text{ mm}$	$s_2 = 51 \text{ mm}$	$s_3 = 52 \text{ mm}$	$s_4 = 53 \text{ mm}$	$s_5 = 54 \text{ mm}$
FM = 0.15	FM = 0.45	FM = 0.80	FM = 0.50	FM = 0.18

1. À quelle position du capteur la région est-elle la mieux mise au point ? Quelle est la profondeur estimée de manière naïve ? (Utiliser $f = 50 \text{ mm}$.)
2. Pourquoi cette estimation naïve est-elle potentiellement imprécise ?
3. Quelle technique permet d'améliorer l'estimation de la profondeur entre les positions discrètes du capteur ?

Exercice 4 :

Comprendre comment la profondeur peut être estimée à partir de deux images en utilisant la disparité.

Code fourni :

```
import cv2
left = cv2.imread('left.jpg', 0)
right = cv2.imread('right.jpg', 0)
stereo = cv2.StereoBM_create(numDisparities=16, blockSize=15)
disparity = stereo.compute(left, right)
cv2.imshow("Disparity", disparity)
```

1. Que observez-vous dans la carte de disparité ?
 - Quelles zones sont plus lumineuses ?
 - Quelles zones sont plus sombres ?
2. Que représente la disparité ?
 - Expliquez la relation entre :
 - Le décalage de pixels
 - La profondeur (distance par rapport à la caméra)
3. Quel est le rôle de numDisparities ?
4. Que contrôle blockSize ?

Exercice 5 :

Calculer une carte de disparité à partir d'images stéréoscopiques et la convertir en une carte de profondeur approximative, puis visualiser les deux.

1. Charger les images gauche et droite en niveaux de gris.
 - Pourquoi utilise-t-on des images en niveaux de gris plutôt qu'en RGB ?
 - Que pourrait-il se passer si les deux images ne sont pas alignées ?

2. Utiliser le Block Matching Stéréo d'OpenCV pour calculer la disparité.

Instructions :

- Utiliser `cv2.StereoBM_create()`
- Essayer :
 - `numDisparities = 64`
 - `blockSize = 15`

3. Que représente la disparité en vision stéréoscopique ?

4. Convertir la disparité en virgule flottante et la mettre à l'échelle correctement.

Indication : OpenCV retourne des valeurs de disparité multipliées par 16

- Pourquoi divise-t-on la disparité par 16 ?
- Que se passe-t-il si on n'effectue pas cette mise à l'échelle ?

5. Remplacer les valeurs de disparité nulles ou négatives.

Indication : La division par zéro doit être évitée

6. Calculer la carte de profondeur. Utiliser la formule : $\text{profondeur} = 1 / \text{disparité}$

7. Quelle est la vraie formule de profondeur en vision stéréoscopique ?

8. Que représentent la longueur focale (f) et la ligne de base (B) ?

9. Normaliser les cartes de disparité et de profondeur dans la plage [0, 255]. Utiliser : `cv2.normalize()`

10. Afficher :

- La carte de disparité
- La carte de profondeur

Côte à côte en utilisant une bibliothèque de visualisation

Exercice 6 :

Utiliser un modèle transformer pré-entraîné pour estimer la profondeur à partir d'une seule image et analyser en quoi l'apprentissage profond diffère de la vision stéréoscopique.

1. Qu'est-ce que l'estimation de profondeur monoculaire ?

2. En quoi est-elle différente de la vision stéréoscopique (caméra gauche-droite) ?

3. Pourquoi cette approche est-elle considérée comme une solution basée sur l'IA plutôt que géométrique ?

4. Charger un modèle pré-entraîné. Nous utilisons un modèle transformer pré-entraîné depuis HuggingFace :

- Modèle : DPT
- Chargé via `pipeline("depth-estimation")`

5. Quel est le rôle d'un modèle pré-entraîné ici ?

6. Pourquoi utilise-t-on des transformers plutôt que des CNNs dans l'estimation moderne de profondeur ?

7. Charger une image depuis :

- Un fichier local OU
- Une URL

8. Pourquoi convertit-on les images au format RGB ?

9. Que se passe-t-il si la résolution de l'image est trop faible ?

10. Exécuter le modèle

11. Extraire :

- `predicted_depth`
- La carte de profondeur `depth`

12. Normaliser les valeurs de profondeur dans [0, 1]

13. Inverser la profondeur : $\text{depthinv} = 1 - \text{depth}$

14. Afficher :

- L'image originale

- La profondeur brute
- La profondeur inversée
- La carte de profondeur colorée

15. Tester sur :

- Une scène intérieure
- Une scène extérieure

Exercice 7 : Reconstruction 3D à partir d'une image unique via Deep Learning

Objectif : Implémenter un pipeline de reconstruction 3D monoculaire en intégrant les Vision Transformers (ViT) avec la rétroprojection géométrique.

Partie 1 : Le moteur d'inférence IA

1 : En utilisant la bibliothèque transformers, écrivez le code pour initialiser un pipeline d'estimation de profondeur.

- Utilisez le modèle : LiheYoung/depth-anything-small-hf.

2 : Une fois qu'une image RGB est chargée via PIL, comment la passer dans le pipeline ? Écrivez la commande pour extraire le résultat et convertir la sortie de profondeur en tableau numpy de type float32.

Partie 2 : Le modèle de caméra (Ingénierie géométrique)

3 : Nous supposons un modèle standard de caméra à trou d'aiguille (Pinhole Camera Model). Si le champ de vision horizontal (FOV) est de 60 degrés, calculez la longueur focale (focal_length) en pixels à partir de la largeur de l'image.

- $f = \text{width} / (2 * \tan(\text{rad}(\text{FOV}/2)))$

Partie 3 : Logique de rétroprojection

5 : En utilisant la formule de rétroprojection, écrivez les expressions permettant de calculer les coordonnées 3D (X, Y, Z) pour chaque pixel.

- Coordonnée X : $(u - c_x) * Z / f$
- Coordonnée Y : $(v - c_y) * Z / f$

(Où c_x et c_y sont le centre de l'image et Z est la valeur de profondeur prédite).

Partie 4 : Visualisation 3D avec Open3D

7 : En utilisant la bibliothèque open3d, écrivez les commandes pour :

1. Initialiser un nouvel objet `o3d.geometry.PointCloud`.
2. Assigner votre tableau Nx3 points à `pcd.points`.
3. Assigner votre tableau Nx3 colors à `pcd.colors`.

8 : Les systèmes de coordonnées en vision par ordinateur ont souvent l'axe Y orienté vers le bas. Écrivez une matrice de transformation 4x4 qui inverse les axes Y et Z afin que la scène apparaisse droite lors du rendu dans la fenêtre `o3d.visualization.draw_geometries`.

Images:

